

1 TypeSpecs Syntax Specification

Annotations	A ::= $\overline{\text{@spec } E}$
Expressions	E ::= $x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T ::= $\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll_::n \gg \mid \ll_::n \gg \mid \ll_::n, _::n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F ::= $\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names.)

Figure 1: TypeSpec

It is noteworthy that the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence. Furthermore, not only is $\text{@spec } \overline{E}$ non-empty sequential elements but also multi-line elements unlike the other comma-separated sequences. That is TypeSpecs only allows one specification on one line.

This specification addresses the most fundamental and atomic types based on the TypeSpecs reference on Elixir documentation. The addressed are, starting from the top type **any()** and the bottom type **none()**, controversially vague concept of field types of record types. While thus enables expressing every existing TypeSpecs type, some might come up with questions on fundamentality with subtype relations or notion of expressions between basic types and literal types. In the following sections, these questions are addressed for clarity with listing the built-in types and the syntactic sugars in Table 1 and Table 2.

1.1 TypeSpecs Basic Types and Literals

TypeSpecs is rather expressive especially by providing the literal types that are also used in the code with the same form. However, this is classic that the higher expressiveness brings the higher complexity and TypeSpecs is not exempt. The common trade-off must be discussed here due to the fact that our ultimate goal is to translate TypeSpecs to Elixir Types. In the perspective of translation, while TypeSpecs allows literals types such as $:k$, $[]$, $\ll$, $(\bar{T} \rightarrow T)$ and $n..n'$, Elixir Types will not support $\ll$, and the range type (i.e., $n..n'$) will not appear to users but will be only used internally. This difference will not allow direct translations for all the types and eventually necessitate approximations of them in Elixir Types.

The first question could arise is from the subtype relationship between **atom()** and $:k$. It is true that $:k$ is a subtype of **atom()** since $:k$ represents every single atom type and **atom()** represents a set of all atom types. Nonetheless, we have to contain $:k$ in the syntax specification. That is because $:k$ is a literal type that could be any single atom type, e.g., **true** and **false** are literals that are used as themselves in the code and also in type specification. Therefore, it is always sound for **atom()** to replace **true** or **false** but not vice versa.

It could be viewed that the range literal (i.e., $n..n'$) expresses **neg_integer()**, **non_neg_integer()** and **pos_integer()**. This idea could be backed by Types and Their Syntax in Erlang documentation that these are built-in types which can be thought defined by $..-1$, $0..$ and $1..$, respectively. But Elixir documentation specifies them as basic types and the provided range syntax in Erlang documentation is not correct use. Provided, it is sufficient to say that these are not subtypes of $n..n'$, hence they are necessary to be in the syntax specification.

1.2 TypeSpecs Built-in Types and Syntactic Sugar

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpecs Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpecs Syntactic Sugars

2 Elixir Types Syntax Specification

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infity..n n..infity { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{ $\dots, \overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() bitstring() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{ $\dots, \overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Types Syntax

2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
<code>boolean()</code>	<code>:true</code> or <code>:false</code>
<code>number()</code>	<code>integer()</code> or <code>float()</code>
<code>non_empty_list(T)</code>	<code>non_empty_list(T, empty_list())</code>
<code>list(T, T')</code>	<code>empty_list()</code> or <code>non_empty_list(T, T')</code>
<code>list(T)</code>	<code>empty_list()</code> or <code>non_empty_list(T, empty_list())</code>
<code>list()</code>	<code>empty_list()</code> or <code>non_empty_list(term(), empty_list())</code>

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
<code>{...}</code>	<code>tuple()</code>
<code>%{...}</code>	<code>open_map()</code>
<code>k : T</code>	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpecs to Elixir Types

$\llbracket \cdot \rrbracket$: `TypeSpecs` $\rightarrow$ Elixir type

Figure 3: Semantic of Translation

3.1 Translation of TypeSpecs Specification into Elixir Annotations

$$\begin{aligned}
\llbracket @spec\ E \rrbracket &= \$ \llbracket E \rrbracket \\
\llbracket @spec\ \bar{E}, @spec\ E' \rrbracket &= \$ \llbracket \bar{E}, E' \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \text{ when } \bar{\alpha} : \bar{T}_3 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases} \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \text{ when } \bar{\alpha}' : \bar{T}'_3 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' = \emptyset \\ S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket[\alpha' := \beta'] \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\beta}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 4: Translation into Annotations

3.2 Translation of TypeSpecs into Elixir Types

```

[[α]] = α
[[T1 | T2]] = [[T1]] or [[T2]]
[[any()]] = term()
[[none()]] = none()
[[atom()]] = atom()
[[ :k ]] = :k
[[pid()]] = pid()
[[port()]] = port()
[[reference()]] = reference()
[[float()]] = float()
[[integer()]] = integer()
[[neg_integer]] = infy..-1
[[non_neg_integer]] = 0..infy
[[pos_integer]] = 1..infy
[[tuple()]] = tuple()
[[{T̄}]] = {[T̄]}
[[%{F1, ..., Fn}]] = map(⊕i=1n[[Fi]])
[[[]]] = empty_list()
[[nonempty_maybe_improper_list(T1, T2)]] = non_empty_list([[T1]], [[T2]])
[[«»]] = bitstring()
[[«_::n»]] = { binary()      if n mod 8 = 0
               bitstring()  otherwise
[[«_::_*n»]] = { binary()      if n mod 8 = 0
               bitstring()  otherwise
[[«_::n, _::_*n»]] = { binary()      if n mod 8 = 0
                      bitstring()  otherwise
[[ (T̄ -> T) ]] = ([T̄] -> [T])
[[ (... -> T) ]] = { fun()      if [[T]] ≈ term()
                    undefined  otherwise
[[n]] = n..n
[[n..n']] = n..n'

```

Figure 5: Translation in Types

3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or `if_set(T)`. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by `Empty` the lists that maps all the non-singleton keytypes to `not_set()`.

Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $K \Rightarrow \tau$ of L_2 such that $K \notin L_1$. Formally:

$$\begin{aligned}
L_1 \oplus [] &= L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1
\end{aligned}$$

Figure 6: Merge in Field Types

We then define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible keytypes), then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

All it remains to do is to define the translation of field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 7.:

$$\begin{aligned}
\llbracket \text{required}(T) \Rightarrow T' \rrbracket &= \begin{cases} [:k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket \text{optional}(T) \Rightarrow T' \rrbracket &= \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

where all K_i 's in I are pairwise distinct.

Figure 7: Translation in Field Types

4 Total Approximating Translation of TypeSpecs into Elixir Types

The translation defined so far is partial since some TypeSpecs types do not have a corresponding Elixir type. This is not a weakness of the Elixir type system, but rather a consequence of the design choices made for Elixir types, to keep them as simple as possible while still being useful. However, it is possible to define a total translation from TypeSpecs types to Elixir types that approximates the original TypeSpecs type. This total translation can be useful in some contexts, for example when we want to ensure that every TypeSpecs type can be represented in Elixir types, even if approximately.

More precisely, the previous translation is undefined in the following cases:

- The type $\dots \rightarrow T$ when the translation of T is not equivalent to $\text{any}()$ (explain why this type is not interesting).
- $\llbracket \text{required}(T) \Rightarrow T' \rrbracket$ and $\llbracket \text{optional}(T) \Rightarrow T' \rrbracket$ are undefined when $\llbracket T \rrbracket$ is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every TypeSpecs type. The extension consists in defining approximations for the cases where the previous translation is undefined.

4.1 Approximation of Top Function Types

The top function type is union of all the functions with every arity to the top type as defined in Figure 8. Since the top function type already takes over all the other function types, it becomes not worthy to have all the other function types that the return type is a subtype of the top type. In the perspective of memory space, it is only costly to define and have all the function types because the maximum arity is 255.

$$\dots \rightarrow \text{any}() \equiv \text{none}() \rightarrow \text{any}() \mid \dots \mid \underbrace{(\text{none}(), \dots, \text{none}())}_{255 \text{ times}} \rightarrow \text{any}()$$

Figure 8: Top Function Type

However, Elixir Types will not directly define `fun()` as Figure 8, due to the triviality of defining the list of function with all the possible arity. While Elixir Types restricts the top type to be only `fun()`, keeping the semantic approach to the top function, TypeSpecs has allowed to specify the return type of the top type which have to be taken into account when translating it. Nevertheless, this leniency in expressiveness means nothing in practice, given that the Dialyzer does not differentiate the return type but only perceive any form of the top types as `...->any()`. These finally lead approximation of the any top function types into `fun()` and `dynamic()`.

$$\llbracket (\dots \rightarrow T) \rrbracket \approx \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{fun}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases}$$

Figure 9: Approximation of Top Function Type

4.2 Approximation of Range Types

$$\begin{aligned} \llbracket \text{neg_integer} \rrbracket &= \text{infy}..-1 \\ \llbracket \text{non_neg_integer} \rrbracket &= 0..\text{infy} \\ \llbracket \text{pos_integer} \rrbracket &= 1..\text{infy} \\ \llbracket n \rrbracket &= n..n \\ \llbracket n..n' \rrbracket &= n..n' \end{aligned}$$

Figure 10: Range Types

4.3 Approximation of Field Types

A field type Approximation is required when two or more than two disjunctive literal types are positioned in left-hand side of field types. That is when

Since fields translation requires merging between each field, it